

The Functional Machine Calculus III: Choice

Early Announcement

Willem Heijltjes¹

*Department of Computer Science
University of Bath
Bath, United Kingdom*

Abstract

The Functional Machine Calculus (Heijltjes 2022) is an extension of the lambda-calculus that preserves confluent reduction and typed termination, while enabling both call-by-name and call-by-value reduction behaviour and encoding the computational effects of mutable higher-order store, input/output, and probabilistic computation. In this note the calculus is extended to capture exception handling and loop constructs.

Keywords: exception handling, lambda-calculus, computational effects

1 Introduction

The Functional Machine Calculus (FMC) [25,5] is a new approach to combining the λ -calculus—as the foundation of functional programming—with computational effects. It takes a view of the λ -calculus as an instruction language for an abstract machine with a single stack in the style of Krivine [38], where *application* is *push*, *abstraction* is *pop*, and *variable* is *execute*. To accommodate effects, the calculus introduces the following two extensions [25].

Locations Multiple stacks on the machine, each named by a *location*, allow the encoding of various effects via push and pop actions: *mutable higher-order store*, as stacks of depth at most one; *input/output*, as pop-only respectively push-only streams; and *probabilities* and *non-determinism* as probabilistically respectively non-deterministically generated streams.

Sequencing The introduction of *sequential composition* and its unit, imperative *skip*, gives control over evaluation behaviour away from strict call-by-name, and allows the encoding of Plotkin’s call-by-value λ -calculus [68], Moggi’s computational metalanguage [58], and Levy’s call-by-push-value [43].

Encoding effects into the generalized operators of the calculus, rather than introducing primitives, means that two key properties of the λ -calculus are preserved.

Confluence Reduction in the FMC is confluent in the presence of effects. This is a consequence of the separation of operational behaviour, which governs the machine, from local reduction behaviour, which is the interaction of consecutive push and pop actions. Reduction equivalence for state then implements the algebraic laws of Plotkin and Power [66].

¹ Email: <mailto:w.b.heijltjes@bath.ac.uk>

Types The FMC can be simply typed, which conveys strong normalization and termination of the machine. This gives a solution to the problem of typing higher-order store: *Landin’s Knot* [39], which encodes recursion via higher-order store, cannot be typed (in its full generality).

This paper introduces a third extension to the FMC, *choice*, to include a wider range of computational behaviours: *constants*, *conditionals*, *data constructors*, *exception handling*, and *loops*. These have in common that, semantically, they are modelled by *sums* or *coproducts*: for example, the Booleans are given by the type $1 + 1$, the error monad is given by the functor $TX = E + X$ for a set of exceptions E , and loops are modelled by taking a map in $A \rightarrow A + B$ to one in $A \rightarrow B$ (looping on A , exiting on B) [8]. Together, these will be referred to as *choice* constructs.

The aim is sixfold. First and second, to preserve *confluence* and *types*: the resulting calculus should support a natural, confluent reduction relation, and a notion of simple types that guarantees termination of the machine and strong normalization of reduction (in the absence of loops). Third, *minimality*: choice constructs should be captured with as few syntactic operators as possible, avoiding any overlap in functionality and minimizing the interactions or reductions governing the semantics of the calculus. Fourth, *operational semantics*: the calculus should continue to be an instruction language for a simple and natural abstract machine. Fifth, *seamless integration*: different effects should combine seamlessly, without requiring lifting operations. Finally, the FMC has a natural *first-order restriction*, where function arguments are restricted to be (first-order) values, not arbitrary terms. The sixth aim is to preserve this restriction, which ensures that choice constructs are independent of the calculus being first-order or higher-order.

The approach has been to reconsider the notion of *choice* from first principles, with the aim of capturing coproducts and the constructs that they model in a simple and natural way, satisfying the six criteria above. This led to three (mostly) standard syntactic constructions, which however interact with the stack in subtle ways to give new and unexpected reduction behaviours. The resulting calculus is in some ways highly familiar, yet simultaneously in other ways novel and surprising.

In contrast with stateful effects, confluence and type safety are expected for exception handling. The main results are to integrate exceptions seamlessly with stateful effects, to support natural operational and denotational semantics, and to capture a wide range of behaviours with an elegant, minimal syntax. This note will discuss the background literature, introduce the selected *choice* constructs from operational considerations, formally define the calculus and its type system, and demonstrate how it captures existing formulations. For simplicity of exposition, the calculus will omit the *locations* modification, and feature only *sequencing* and *choice*. Proofs are incomplete at the time of writing, and hence omitted, leaving the intended theorems as conjectures.

2 Background

The study of effects in the λ -calculus is broad and varied. The overarching problem is that effects are *sequential* when modelled through global updates, while the λ -calculus is fundamentally *denotational*, governed by an equational theory. The contradiction manifests concretely as the loss of *confluence*, so that the choice of reduction strategy becomes salient, and syntactic control over evaluation behaviour becomes necessary. Semantic reasoning about programs, a main strength of the λ -calculus, is severely impaired—and much research has the objective of restoring it. To quote Filinski in 2011:

Yet few would confidently claim that programs with computational effects are by now as well understood, and as thoroughly supported by formal reasoning techniques, as types and terms in purely functional settings. — Filinski [21]

Since Landin’s seminal work [39,40,41] most practical functional programming languages employ a call-by-value (cbv) semantics, as formalized by Plotkin [68], where call-by-name (cbn) behaviour is forced locally with *thunks*. However, for reasoning about programs a cbn-semantics is desirable. The inefficiency of cbn may be overcome by *graph reduction* [84] (or *lazy evaluation*, or *call-by-need* [81,64,3]), but the imposed notion of sequentiality is opaque, and a semantic approach to effects becomes more urgent.

Among various proposals, such as taming destructive updates through *interference control* [73,60] or *uniqueness types* [76], and explicitly typing effects in *type and effect* systems [22,34,80,59], Moggi’s account of effects as monads [58] became prominent. It casts sequentiality as composition in the Kleisli category

of a monad [49], which may then be presented as sequential programming through syntactic sugar, such as Haskell’s *do*-notation.

However, monads do not compose, raising the issue of how to combine multiple effects. Haskell 98 resorts to cumbersome stacks of *monad transformers* [46] (their limitations are discussed in [37]). One solution is to take sums of monads [48]; another, added in Haskell 2010, is to weaken monads to *applicative* (or *monoidal*) functors [52], which do compose. Both however need intrusive *lifting* functions to access different effects, suggesting that the problem is not merely with compositionality, but instead with explicit composition as such.

Levy’s *call-by-push-value* (cbpv) [43] unifies effects in a single monad (not unlike the IO monad in Haskell [33]) formed by an adjunction separating *values* and *computations*. It arose in the context of two closely related alternatives. Power, Robinson, and Thielecke sought to connect Hasegawa’s κ -calculus [24] with a semantics in *premonoidal* categories [72,71], which impose sequentiality on composition in a monoidal category by weakening its naturality laws—a semantics later applied to cbpv [45]. This was mirrored by Hughes’ Arrows for Haskell [29,47], which generalise Kleisli composition of monads in a similar way [4]. The similarities between cbpv and κ -calculus are striking: both feature sequentiality through an operational semantics in a stack machine, a decomposition of computations and values, and an embedding of cbn- and cbv- λ -calculus. It was cbpv that gained traction, being extended [15] and rediscovered [16] through linear logic and forming the basis of *effect handlers* [67].

Effect handlers [67,36,85,50] combine cbpv with *algebraic effects* [66,31,30], a view of effects as algebraic theories that enables equational reasoning with effect operators, though not reduction. This is implemented by restricting *continuations* to enforce *sequentiality* (a restriction that is sometimes relaxed), generalising *exception handlers* [6].

Continuations as a means of controlling sequential behaviour date back to ALGOL60 and Landin’s J-operator [40] used to model its *go to* construct (Reynolds provides an overview of this early history [74]), subsequently developed in the context of the cbv- λ -calculus [17,18]. Three developments then gave continuations a central place in programming language theory and practice: the discovery of a Curry–Howard relation with classical logic [23], soon after expressed in Parigot’s $\lambda\mu$ -calculus [62,78]; their usefulness in compilers [2]; and the characterization of the continuation monad as universal among effect monads [19,20].

The distinction between continuations and *exception handling* is wafer-thin in a higher-order language, and exceptions in λ -calculus are frequently studied through continuations [11,61,7,9,35,42,82]. Subtle differences in reduction arise depending on the choice of semantics: in classical logic and $\lambda\mu$ -calculus, or in intuitionistic coproducts as the *exceptions-as-values* approach generally models them [83,77,32], or in intuitionistic continuations as do the exception handlers [6,44] that inspired effect handlers [67].

The Functional Machine Calculus itself [25,5] originates in a confluent probabilistic λ -calculus [10], and while developed independently, many individual aspects are familiar. *Sequencing*, which locates sequentiality in the operational semantics of a stack machine, appeared as κ -calculus [24,71], in compiler theory [12], and in practice as *concatenative programming* [26,63,54]. *Locations* are akin to the *channels* of process calculi [28,55,57], and their encoding of mutable store is that of Haskell’s *MVar* mutable variables for shared-memory concurrency [65]. A close overall relative are *action calculi* [53,56]. The FMC as a whole is nevertheless novel and surprising, as the introduction sets out.

The *choice* extension in this note uses standard syntactic constructions for exception handling, with subtly novel reduction behaviour due to the interaction with the stack. It takes a semantics in intuitionistic sums, and avoids swerving into continuations-territory by the design requirements of *simple types* and *first-order restriction*. Key inspirations are compiler optimizations through sequent calculus and sequentially-used continuations [13,51], from which the terminology of *jumps* and *joins* is adopted. The *loop* construct is believed to be a new syntactic interpretation of the standard semantics for iteration [8], dual to recursion [75].

3 Development

The FMC extends the λ -calculus with *sequential composition* ($M;N$) and *skip* ($\star$), its unit, the empty instruction sequence. In an imperative context, *skip* signifies *successful termination*. The FMC introduces this concept into the Krivine Machine (which traditionally halts with an abstraction and empty stack, or a variable), and it is a key feature underlying many aspects of the calculus, including the design of the

type system, the semantics, and the termination and normalization proofs [25,5].

The central idea of this paper is to generalise *skip* to allow not just *success*, but arbitrary notions of *failure*, i.e. exceptions or exit codes. This is modeled by a set of *jumps* $\mathcal{J}$, with *skip* ($\star \in \mathcal{J}$) the default value. As a term, a jump j corresponds to raising an exception. The machine may terminate with any jump $j \in \mathcal{J}$ as exit status, plus a stack of return values. Semantically, a term returns a coproduct over stacks, indexed in a finite set $J \subset \mathcal{J}$.

Standard sequencing $M; N$ evaluates M to $\star$, and then continues with N , expressed in the reduction rule $\star; N \rightarrow N$. When M may evaluate to any jump j , sequencing is likewise generalized by making it conditional on a particular jump, to a *join* $M; j \rightarrow N$: if M terminates in j then continue as N (and otherwise discard N). This makes $M; j \rightarrow N$ a standard *try/catch* construct, try M catch j N . The behaviour is formalized by the reduction rules $j; j \rightarrow N \rightarrow N$ and $i; j \rightarrow N \rightarrow i$ (where $i \neq j$). The original sequencing $M; N$ becomes an abbreviation for the default case $M; \star \rightarrow N$.

The construction lets us cast constants as jumps, for example the Booleans as $\top$ and $\perp$ with the conditional if B then M else N given by two joins as $B; \top \rightarrow M; \perp \rightarrow N$ (which associates as $(B; \top \rightarrow M); \perp \rightarrow N$, and assumes that both M and N exit with *skip*). Our *join* is then also a standard *case switch*:

$$\text{switch } M \text{ (case } j_1 : N_1 \dots \text{case } j_n : N_n)$$

becomes $M; j_1 \rightarrow N_1 \dots ; j_n \rightarrow N_n$. Such switches may have different *fall-through* policies (whether after a matching case also subsequent cases are considered), often with a *break* or *return* keyword to exit; it will come as no surprise that these are readily modelled with jumps and joins, where *break* is another jump.

A subtle but key aspect is the difference between returning a constant as a value on the stack, and evaluating it as a jump. A standard function returning a Boolean should be interpreted in the former way, pushing $\top$ or $\perp$ onto the stack and exiting with $\star$, and indeed terms are generally expected to exit successfully. To *evaluate* a constant, triggering subsequent join constructs, is the domain of conditionals, case switches, and exception handling.

Joins as conditional sequencing give a clean approach to *iteration*. A loop M^j evaluates M , then loops on j or exits on other jumps. This is implemented by the reduction $M^j \rightarrow M; j \rightarrow M^j$. A standard *do-while* loop (do M while B) with a Boolean B becomes $(M; B)^\top; \perp \rightarrow \star$, with the expected behaviour: it reduces to $M; B; \top \rightarrow (M; B)^\top; \perp \rightarrow \star$, which is the interpretation of

$$M; \text{ if } B \text{ then (do } M \text{ while } B) \text{ else } \star.$$

As before, control flow through *break* or *return* keywords may be modelled by viewing these as jumps, and positioning corresponding *join* constructs outside the loop.

On an abstract machine, it is standard to manage sequencing with a *continuation stack*, where $M; N$ pushes N , and *skip* pops. The FMC, in the formulation used here, introduces this into the Krivine Machine. The generalization to jumps is captured on the machine by modifying the continuation stack to hold *conditional terms*: $M; j \rightarrow N$ pushes $j \rightarrow N$, while j inspects the head $i \rightarrow N$ and pops N if $i = j$, discarding it otherwise. A loop M^j pushes $j \rightarrow M^j$ and continues as M . The three constructs thus represent natural instructions on the machine, with j representing a forward jump and M^j a controlled backward jump.

Types for the FMC indicate the input and output stacks of successful machine evaluation: a term of type $\sigma_n \dots \sigma_1 \Rightarrow \tau_1 \dots \tau_m$ expects an input stack of terms typed σ_1 through σ_n and produces a return stack with types $\tau_1 \dots \tau_m$. Semantically, terms are morphisms in a category where objects are type vectors $\vec{\tau} = \tau_1 \dots \tau_m$ (assuming a single stack for simplicity; with multiple stacks, objects are families of type vectors). With jumps, return types become sums of jump-indexed type vectors, each written $\vec{\tau}.j$, rendering types as

$$\vec{\sigma} \Rightarrow (\vec{\tau}_1.j_1 + \dots + \vec{\tau}_n.j_n).$$

For example, using 1 for the empty stack the type for Booleans becomes $1 \Rightarrow 1.\top + 1.\perp$. As this illustrates, jumps (as terms) are the *injections* of the coproduct.

A join $M; j \rightarrow N$ composes on the j -indexed summand; for example for a Boolean B and $M : 1 \Rightarrow \vec{\tau}.\star$ we have $B; \top \rightarrow M : 1 \Rightarrow \vec{\tau}.\star + 1.\perp$. Codiagonals (the natural maps $A + A \rightarrow A$) are given implicitly by two joins with the same return type *and* jump. For example, if M and N are both typed $1 \Rightarrow \vec{\tau}.\star$ then the conditional $B; \top \rightarrow M; \perp \rightarrow N$ will have the type $1 \Rightarrow \vec{\tau}.\star$ as well.

The types for *loops* follow the standard semantics of iteration. Since M^j iterates on j and exits on other

jumps, the return stack of M for the jump j must match its input stack, and this summand is removed for M^j . For example, if $M : \tau \Rightarrow \tau.\star$ and B is a Boolean, then $M ; B : \tau \Rightarrow \tau.\top + \tau.\perp$, and $(M ; B)^\top : \tau \Rightarrow \tau.\perp$; i.e. if this loop terminates, it will be with a return value of type τ and exit status $\perp$.

The three constructors *jump* j , *join* $M ; j \rightarrow N$, and *loop* M^j thus form a concise internal language for coproducts with iteration. Next, we consider these in the context of the FMC, as an extension to the λ -calculus viewed through the lens of the (simplified) Krivine machine.

4 The FMC with Choice

The FMC adopts a syntax that emphasises the machine perspective, writing *application* $M N$ as $[N].M$ to mean *push* N and continue with M , and *abstraction* $\lambda x.M$ as $\langle x \rangle.M$ for *pop*, to continue with M where the popped term replaces x . The full FMC [25] parameterizes the *push* and *pop* operations in a set of *locations* $\{a, b, c, \dots\}$, each indicating a stack on the machine, as $[N]a.M$ and $a\langle x \rangle.M$. This extension is orthogonal to *choice* as considered here, and for simplicity is omitted. The constructions are thus introduced into the plain λ -calculus, but they generalise *sequencing* $M ; N$ and *skip* $\star$ of the fragment of the FMC called the *sequential λ -calculus*.

Definition 4.1 Let x, y, z range over a countable set of *variables* $\mathcal{V}$, and i, j, k over a countable set of *jumps* $\mathcal{J}$ that includes the distinguished element $\star$. *Terms* are given by the following grammar.

$$M, N, P, Q ::= x \mid [N].M \mid \langle x \rangle.M \mid j \mid N ; j \rightarrow M \mid M^j$$

The constructors are: a *variable* x , an *application* or *push action* $[N].M$, an *abstraction* or *pop action* $\langle x \rangle.M$ which binds x in M , a *jump* j , a *join* $N ; j \rightarrow M$, and a *loop* M^j . The free variables of a term are written $\text{fv}(M)$, and capture-avoiding substitution of N for x in M is written $\{N/x\}M$.

Definition 4.2 The *reduction rules* are:

$$\begin{array}{ll} \text{Beta} & [N].\langle x \rangle.M \rightarrow \{N/x\}M \\ \text{Select} & j ; j \rightarrow M \rightarrow M \\ \text{Skip} & i ; j \rightarrow M \rightarrow i \quad (i \neq j) \\ \text{Unroll} & M^j \rightarrow M ; j \rightarrow M^j \\ \text{Prefix (pop)} & \langle x \rangle.N ; j \rightarrow M \rightarrow \langle x \rangle.(N ; j \rightarrow M) \quad (x \notin \text{fv}(M)) \\ \text{Prefix (push)} & [P].N ; j \rightarrow M \rightarrow [P].(N ; j \rightarrow M) \end{array}$$

The *reduction relation* $\rightarrow$ is given by closing the above rules under any context. The reflexive–transitive closure of $\rightarrow$ is written $\Rightarrow$.

The *beta*-reduction rule is that of the λ -calculus. The *select* and *skip* rules implement a jump from j to the next join $j \rightarrow M$, skipping others in between. The two *prefix* rules govern the interaction between the λ -calculus and sequencing. They implement the notion that *push* and *pop* actions are *sequential* via the (standard) interaction of prefixing with concatenation.

Proposition 4.3 The normal forms N_0 of reduction $\rightarrow$ are given by the following grammars.

$$\begin{array}{ll} N_0 ::= & \langle x \rangle.N_0 \mid N_1 \\ N_1 ::= & [N_0].N_1 \mid N_2 \mid j \\ N_2 ::= & N_2 ; j \rightarrow N_0 \mid x \end{array}$$

The machine will evaluate a term in the context of an *argument stack* $S = M_1 \dots M_n$, holding terms, and a *continuation stack* $K = (j_1 \rightarrow M_1) \dots (j_n \rightarrow M_n)$, holding conditional continuations $j \rightarrow M$ to be used on a jump j and discarded on any other jump.

Definition 4.4 The *states* of the abstract machine are triples (S, M, K) of an *argument stack* S , a term M , and a *continuation stack* K , where both stacks are defined as follows.

$$S, T ::= \varepsilon \mid S M \quad K, L ::= (j \rightarrow M) K \mid \varepsilon$$

The *transitions* of the machine are given by the following top-to-bottom rules.

$$\begin{array}{c}
 \frac{(S, [N].M, K)}{(S, N, M, K)} \quad \frac{(S, N; j \rightarrow M, K)}{(S, N, (j \rightarrow M)K)} \\
 \\
 \frac{(S, N, \langle x \rangle.M, K)}{(S, \{N/x\}M, K)} \quad \frac{(S, j, (j \rightarrow M)K)}{(S, M, K)} \\
 \\
 \frac{(S, M^j, K)}{(S, M, (j \rightarrow M^j)K)} \quad \frac{(S, i, (j \rightarrow M)K)}{(S, i, K)} \quad (i \neq j)
 \end{array}$$

A *run* of the machine is a sequence of steps written as below left. A run is *complete* if it terminates with a jump j as term and an empty continuation stack, as below right.

$$\frac{(S, M, K)}{(T, N, L)} \quad \frac{(S, M, K)}{(T, j, \varepsilon)}$$

The abstract machine defines the small-step operational semantics. A big-step semantics is given by composing complete runs, as follows.

Definition 4.5 The *evaluation* relation $S, M \Downarrow T, j$ is defined inductively as follows.

$$\begin{array}{c}
 \frac{S, N, M \Downarrow T, j}{S, [N].M \Downarrow T, j} \quad \frac{S, M \Downarrow R, i \quad R, N \Downarrow T, j}{S, M; i \rightarrow N \Downarrow T, j} \quad \frac{S, M \Downarrow T, j}{S, M; i \rightarrow N \Downarrow T, j} \quad (i \neq j) \\
 \\
 \frac{S, \{N/x\}M \Downarrow T, j}{S, N, \langle x \rangle.M \Downarrow T, j} \quad \frac{S, M \Downarrow R, i \quad R, M^i \Downarrow T, j}{S, M^i \Downarrow T, j} \quad \frac{S, M \Downarrow T, j}{S, M^i \Downarrow T, j} \quad (i \neq j)
 \end{array}$$

Proposition 4.6 *Small-step and big-step operational semantics agree:*

$$\frac{(S, M, \varepsilon)}{(T, j, \varepsilon)} \iff S, M \Downarrow T, j$$

To relate reduction and machine evaluation, the reduction relation $\rightarrow$ is extended to argument stacks by reducing on a single term: $T, M \rightarrow T, N$ if $M \rightarrow N$.

Proposition 4.7 *Reduction commutes with machine evaluation: if $S \twoheadrightarrow R$, $M \twoheadrightarrow N$, and $S, M \Downarrow T, j$ then there is a stack U such that $T \twoheadrightarrow U$ and $R, N \Downarrow U, j$.*

Conjecture 4.8 *Reduction $\rightarrow$ is confluent.*

5 Types

Types for the sequential λ -calculus are as follows: stacks are typed $S : \vec{\sigma}$ by a vector of types, and terms are typed $M : \vec{\sigma} \Rightarrow \vec{\tau}$ indicating the expected input and output stacks for a complete run of M on the machine. To accommodate *choice*, the return type is generalised to a sum of jump-indexed type vectors, each written $\vec{\tau}.j$, to give the following type structure: $M : \vec{\sigma} \Rightarrow (\vec{\tau}_1.j_1 + \dots + \vec{\tau}_n.j_n)$.

Definition 5.1 *Types* are given as follows, where $I, J \subset \mathcal{J}$ denote a finite set of jumps.

$$\begin{array}{ll}
 \text{Types:} & \rho, \sigma, \tau ::= \vec{\sigma} \Rightarrow \vec{\tau}_J \\
 \text{Stack types:} & \vec{\tau} ::= \tau_1 \dots \tau_n \\
 \text{Choice types:} & \vec{\tau}_J ::= \{\vec{\tau}_j \mid j \in J\}
 \end{array}$$

Variable:	$\overline{\Gamma, x : \tau \vdash x : \tau}$
Application:	$\frac{\Gamma \vdash N : \rho \quad \Gamma \vdash M : \rho \vec{\sigma} \Rightarrow \vec{\tau}_J}{\Gamma \vdash [N].M : \vec{\sigma} \Rightarrow \vec{\tau}_J}$
Abstraction:	$\frac{\Gamma, x : \rho \vdash M : \vec{\sigma} \Rightarrow \vec{\tau}_J}{\Gamma \vdash \langle x \rangle.M : \rho \vec{\sigma} \Rightarrow \vec{\tau}_J}$
Jump:	$\overline{\Gamma \vdash j : 1 \Rightarrow 1.j}$
Join:	$\frac{\Gamma \vdash N : \vec{\rho} \Rightarrow \vec{\tau}_I + \vec{\sigma}.j \quad \Gamma \vdash M : \vec{\sigma} \Rightarrow \vec{\tau}_I}{\Gamma \vdash N ; j \rightarrow M : \vec{\rho} \Rightarrow \vec{\tau}_I}$
Loop:	$\frac{\Gamma \vdash M : \vec{\sigma} \Rightarrow \vec{\tau}_I + \vec{\sigma}.j}{\Gamma \vdash M^j : \vec{\sigma} \Rightarrow \vec{\tau}_I}$
Stack expansion:	$\frac{\Gamma \vdash M : \vec{\rho} \Rightarrow \vec{\tau}_J}{\Gamma \vdash M : \vec{\rho} \vec{\sigma} \Rightarrow (\vec{\sigma} \vec{\tau})_J}$
Sum expansion:	$\frac{\Gamma \vdash M : \vec{\rho} \Rightarrow \vec{\tau}_J}{\Gamma \vdash M : \vec{\rho} \Rightarrow \vec{\sigma}_I + \vec{\tau}_J}$
Stack types:	$\frac{\{\Gamma \vdash M_i : \tau_i\}_{i \leq n}}{\Gamma \vdash M_1..M_n : \tau_1 \dots \tau_n}$

Fig. 1. Typing rules

Observe that the identity function on a stack of two elements is $\langle x \rangle. \langle y \rangle. [y]. [x]. \star$, while $\langle x \rangle. \langle y \rangle. [x]. [y]. \star$ is the symmetry. The order of input types follows that of the abstractions, so that the type for the above identity is $\sigma\tau \Rightarrow \tau\sigma$ where $x : \sigma$ and $y : \tau$, and in general identity types are $\vec{\tau} \Rightarrow \vec{\tau}$ where $\vec{\tau}$ is the reverse vector to $\vec{\tau}$. Further notation includes: concatenation of type vectors by juxtaposition $\vec{\sigma}\vec{\tau}$, and the empty vector as 1 . The disjoint union of choice types as families is written $\vec{\sigma}_I + \vec{\tau}_J$ where $I \cap J = \emptyset$, and $\vec{\tau}.j$ denotes the singleton family consisting of the vector $\vec{\tau}$ indexed by j . A choice type may then be written $\vec{\tau}_1.j_1 + \dots + \vec{\tau}_n.j_n$ where the jumps are disjoint.

Semantically, types $\vec{\sigma} \Rightarrow \vec{\tau}_J$ are function spaces $\vec{\sigma} \rightarrow \vec{\tau}_J$, stack types $\tau_1 \dots \tau_n$ are products $\tau_1 \times \dots \times \tau_n$, and choice types $\{\vec{\tau}_j \mid j \in J\}$ are indexed sums $\sum_{j \in J} (\vec{\tau}_j)$. *Locations* are indexed products [5], giving the duality between state and exceptions observed in [14].

Definition 5.2 A *typing judgement* $\Gamma \vdash M : \tau$ assigns a term M the type τ in the context Γ , where a context is a finite function from variables to types written as a comma-separated list $x_1 : \tau_1, \dots, x_n : \tau_n$. A *stack typing judgement* $\Gamma \vdash S : \vec{\tau}$ assigns a stack the type vector $\vec{\tau}$. The *typing rules* for the calculus are given in Figure 1.

Proposition 5.3 *Subject reduction holds: if $\Gamma \vdash M : \tau$ and $M \rightarrow N$ then $\Gamma \vdash N : \tau$.*

Conjecture 5.4 *Types guarantee machine termination: for a loop-free, closed, typed term $\vdash M : \vec{\sigma} \Rightarrow \vec{\tau}_J$ and stack $\vdash S : \vec{\sigma}$ there is a jump $j \in J$ and stack $\vdash T : \vec{\tau}_j$ with $\vec{\tau}_j \in \vec{\tau}_J$ such that $S, M \Downarrow T, j$.*

Conjecture 5.5 *Types guarantee strong normalization: a loop-free, typed term $\Gamma \vdash M : \tau$ does not have an infinite reduction path in $\rightarrow$.*

6 Discussion

The careful design of the choice constructions for the FMC has subtle implications, which manifest more clearly in the context of the stack machine, the operational semantics of the calculus. We consider these here.

First, we consider data constructors. Standard case constructs such as are found in ML or Haskell are a direct interpretation of the disjunction elimination rule of natural deduction,

$$\text{case } M \text{ of } (\text{inl } x \rightarrow N ; \text{inr } y \rightarrow P)$$

generalised to indexed sums (data types with constructors) and multiple parameters. As does ML [1], the present approach unifies data constructors and exceptions, and renders the above as follows.

$$M ; \text{inl} \rightarrow \langle x \rangle . N ; \text{inr} \rightarrow \langle y \rangle . P$$

Instead of parameterizing the constructors as $\text{inl } x$ and $\text{inl } y$, the two joins use a regular *pop*, $\langle x \rangle . N$ or $\langle y \rangle . P$, to collect the return value from M . This is possible due to the nature of the FMC as a stack-based calculus, where M leaves its return value on the stack: an application to a constructor $\text{inl } Q$ translates directly as $[Q].\text{inl}$, pushing Q to the stack and exiting with inl . Construction and deconstruction generalize to any number of arguments, simplyfying the syntax from a *sequence* of cases each with a *sequence* of parameters, to the single constructor $M ; j \rightarrow N$. The correctness of the interpretation is guaranteed by typing, which ensures that N does not exit with inr which would incorrectly trigger P in the example above, or in the untyped case by strictly jumps into *constants/constructors* and *exceptions*, and maintaining that constants and constructors are returned on the stack.

The fall-through aspect of joins makes them a potential solution to the problem of *extensible data types* [79] (adding further constructors to an existing data type). However, the present exposition does not include *inductive* types, or *recursion* to compute with them, so at the moment this is only an interesting future direction.

Secondly, we will look at exception handling mechanisms. Though there are many variations, the underlying principle is the same: a computation may *raise* or *throw* an exception, which is then *caught* by some *handler*. First, we discuss the formulation by De Groote [11], for it raises an important issue. It considers exception handling in ML, and proceeds to give a variation on it via classical logic; we will refer to the original ML variant as *coproduct-style*, and that of classical logic, *continuation-style*. The syntax is as follows, slightly simplified and using M, N for terms and e for exceptions.

$$\text{raise } M \qquad M \text{ handle } (ex) \Rightarrow N$$

The key distinction between coproduct-style and continuation-style exception handling is found in the rule for handling a return value (that is, not an exception), where continuation-style requires that e is not free in V , but coproduct-style does not:

$$V \text{ handle } (ex) \Rightarrow N \rightarrow V$$

That is, continuation-style considers exceptions to be *variables*, where the handler binds e in V . Accordingly, the handling construct may be interpreted with a let-binding as follows:

$$M \text{ handle } (ex) \Rightarrow N \quad \mapsto \quad \text{let } ex = N \text{ in } M$$

By contrast, coproduct-style handling treats exceptions as *constructors* or *constants*. Handling may then be interpreted by a case construct, as follows, where the variable y matches every other case than the exception e , returning it unchanged.

$$M \text{ handle } (ex) \Rightarrow N \quad \mapsto \quad \text{case } M \text{ of } (ex \rightarrow N ; y \rightarrow y)$$

Types for the two encodings clarify the difference: for continuation style, in $\text{let } ex = N \text{ in } M$ we expect $e : A \rightarrow B$ for $x : A$, $N : B$, and $M : B$, giving the continuation type $(A \rightarrow B) \rightarrow B$ to $\lambda e.M$. For coproduct style, in $\text{case } M \text{ of } (ex \rightarrow N ; y \rightarrow y)$ we have $M : A + B$ for $x : A$ and $N : B$, with e the left injection of the coproduct. The two styles are thus related by the standard higher-order encoding of a coproduct $A + B$ as $(A \rightarrow C) \rightarrow (B \rightarrow C) \rightarrow C$.

Exception handling in the literature generally takes one these two forms: interpretations in $\lambda\mu$ -calculus [61,7,9,82] are naturally continuation-style, while the exceptions-as-values approach [83,77,32], including the exception monad $TX = E + X$, is in coproduct style. The FMC with Choice is in coproduct

style, with the following encoding $(-)_t$.

$$(M \text{ handle } (e x) \Rightarrow N)_t = M_t ; e \rightarrow \langle x \rangle . N_t$$

This extends a standard interpretation of Plotkin's $\text{cbv-}\lambda$ -calculus [68] into stack calculi [12, 71, 25], here formulated as $(-)_v$ and $(-)_t$.

$$\begin{array}{ll} \text{Values } V, W : & x_v = x \\ & (\lambda x.M)_v = \langle x \rangle . M_t \\ \text{Terms } M, N : & V_t = [V_v].\star \\ & (M N)_t = M_t ; \langle v \rangle . (N_t ; v) \quad (v \text{ fresh}) \end{array}$$

The interpretation of an application $M N$ is to evaluate M_t to a value $[V_v].\star$; then to pick up V_v as v ; evaluate N_t to $[W_v]$; and evaluate v with W_v as its first argument on the stack. The interpretation extends to simple types as follows.

$$o_v = 1 \Rightarrow 1.\star \quad (A \rightarrow B)_v = A_v \Rightarrow B_v.\star \quad A_t = 1 \Rightarrow A_v.\star$$

Exceptions e are typed as injections, $e : 1 \Rightarrow 1.e$, and when applied to a term $M_t : 1 \Rightarrow A_v.\star$ the type becomes $(e M)_t \rightarrow M_t ; e : 1 \Rightarrow A_v.e$. The *raise* construct is interpreted by the *sum expansion* typing rule: if $M_t : 1 \Rightarrow A_v.e$ then $\text{raise } M$ translates to $M_t : 1 \Rightarrow A_v.e + B_v.\star$ for some type B_v .

The ML-style reduction rules [11, Table 1] are then simulated in the FMC, preserving types. Four cases are demonstrated below (with the first two slightly adjusted; in the original *raise* alone signifies an exception, but the translation requires an explicit e).

$$\begin{array}{ll} V(\text{raise}(e W)) \rightarrow \text{raise}(e W) & [V_v].\star ; \langle x \rangle . ([W_v].e ; x) \rightarrow [W_v].e \\ (\text{raise}(e V))W \rightarrow \text{raise}(e V) & [V_v].e ; \langle x \rangle . ([W_v].\star ; x) \rightarrow [W_v].e \\ V \text{ handle } (e x) \Rightarrow N \rightarrow V & [V_v].\star ; e \rightarrow \langle x \rangle . N_t \rightarrow [V_v].\star \\ \text{raise } e V \text{ handle } (e x) \Rightarrow N \rightarrow \{V/x\}N & [V_v].e ; e \rightarrow \langle x \rangle . N_t \rightarrow \{V_v/x\}N_t \end{array}$$

The relation with *algebraic effects* [69] and *effect handlers* [67] is not yet fully characterised. It is clear that effect handlers should be considered as continuation-style, and a handler understood as a binding construct for effect operations. A tentative interpretation is the following, where operators op are variables and a handler H is a *prefix* of push- and pop-actions, binding in N .

$$\begin{array}{ll} \text{op } V(x.M) &= [V].\text{op} ; \langle x \rangle . M \\ N \text{ handle } H \text{ to } x.M &= H.N ; \langle x \rangle . M \\ (\text{op } x k \rightarrow N) + H &= [\langle x \rangle . \{\star/k\}N].\langle \text{op} \rangle . H \end{array}$$

Note that the continuation k in N is discarded, and replaced with *skip*, and correspondingly the continuation K in $\text{op } V K$ is composed sequentially, rather than passed as an argument. This interprets handlers directly in their *generic* form [70, 67]. The interpretation is correct for handlers satisfying the requisite law.

$$\text{op } V(x.M) ; K = \text{op } V(x.(M; K))$$

Shallow handlers [27] superficially appear closer to coproduct-style, since handlers become *affine* (used at most once). However, for subtle reasons, reductions are not fully compatible with the expected interpretation in *choice* constructs. This would be as below, where an effect operator is a *jump* and a handler H is a postfix sequence of *joins*. The discrepancy needs further work to fully understand.

$$\begin{array}{ll} \text{op } V &= [V].\text{op} \\ N \text{ handle } H &= N ; H \\ H + (\text{op } x k \rightarrow N) &= H ; \text{op} \rightarrow \langle x \rangle . \{\star/k\}N \\ \text{return } x \rightarrow N &= \star \rightarrow \langle x \rangle . N \end{array}$$

7 Conclusion

This note has given an overview of current progress on the *choice* extension for the Functional Machine Calculus. At present the design is complete, proofs are expected, and a start has been made on a close comparison with previous work. These are promising: the calculus is expected to fulfil the six aims given in the introduction (*confluence*, *types*, *minimality*, *operational semantics*, *seamless integration*, and *first-order restriction*), and the discussion in the previous section is starting to give an interesting and revealing perspective on past work. Clearly, this is only a starting point, and there is much more to investigate.

Acknowledgements

The author is greatly indebted to Nicolas Wu for explaining effect handlers.

References

- [1] Appel, A., D. MacQueen, R. Milner and M. Tofte, *Unifying exceptions with constructors in standard ml*, Technical Report ECS-LFCS-88-55, Laboratory for Foundations of Computer Science, Computer Science Department, Edinburgh University (1988).
- [2] Appel, A. W., *Compiling with Continuations*, Cambridge University Press (1992), ISBN 0-521-41695-7.
- [3] Ariola, Z. M., M. Felleisen, J. Maraist, M. Odersky and P. Wadler, *The call-by-need lambda calculus*, in: R. K. Cytron and P. Lee, editors, *Conference Record of POPL'95: 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 233–246, ACM Press (1995).
<https://doi.org/10.1145/199448.199507>
- [4] Atkey, R., *What is a categorical model of arrows?*, *Electronic Notes in Theoretical Computer Science* **229**, pages 19–37 (2011).
<https://doi.org/10.1016/j.entcs.2011.02.014>
- [5] Barrett, C., W. Heijltjes and G. McCusker, *The Functional Machine Calculus II: Semantics*, in: B. Klin and E. Pimentel, editors, *31st EACSL Annual Conference on Computer Science Logic (CSL 2023)*, volume 252 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 10:1–10:18, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2023).
<https://doi.org/10.4230/LIPIcs.CSL.2023.10>
- [6] Benton, N. and A. Kennedy, *Exceptional syntax*, *Journal of Functional Programming* **11**, pages 395–410 (2001).
<https://doi.org/10.1017/S0956796801004099>
- [7] Bierman, G. M., *A computational interpretation of the lambda-mu-calculus*, in: L. Brim, J. Gruska and J. Zlatuska, editors, *Proc. 23rd International Symposium on Mathematical Foundations of Computer Science (MFCS'98)*, volume 1450 of *Lecture Notes in Computer Science*, pages 336–345, Springer (1998).
<https://doi.org/10.1007/BFB0055783>
- [8] Bloom, S. L. and Z. Ésik, *Iteration Theories - The Equational Logic of Iterative Processes*, EATCS Monographs on Theoretical Computer Science, Springer (1993), ISBN 978-3-642-78036-3.
<https://doi.org/10.1007/978-3-642-78034-9>
- [9] Crolard, T., *A confluent lambda-calculus with a catch/throw mechanism*, *Journal of Functional Programming* **9**, pages 625–647 (1999).
<https://doi.org/10.1017/S0956796899003512>
- [10] Dal Lago, U., G. Guerrieri and W. Heijltjes, *Decomposing probabilistic lambda-calculi*, in: J. Goubault-Larrecq and B. König, editors, *Foundations of Software Science and Computation Structures*, volume 12077 of *LNCS*, pages 136–156, Springer International Publishing, Cham (2020), ISBN 978-3-030-45231-5.
https://doi.org/10.1007/978-3-030-45231-5_8
- [11] de Groote, P., *A simple calculus of exception handling*, in: M. Dezani-Ciancaglini and G. D. Plotkin, editors, *Proc. Second International Conference on Typed Lambda Calculi and Applications (TLCA '95)*, volume 902 of *Lecture Notes in Computer Science*, pages 201–215, Springer (1995).
<https://doi.org/10.1007/BFB0014054>
- [12] Douence, R. and P. Fradet, *A systematic study of functional language implementations*, *ACM Transactions on Programming Languages and Systems* **20**, pages 344–387 (1998).

- [13] Downen, P., L. Maurer, Z. M. Ariola and S. Peyton Jones, *Sequent calculus as a compiler intermediate language*, in: J. Garrigue, G. Keller and E. Sumii, editors, *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 74–88, ACM (2016).
<https://doi.org/10.1145/2951913.2951931>
- [14] Dumas, J., D. Duval, L. Fousse and J. Reynaud, *A duality between exceptions and states*, *Mathematical Structures in Computer Science* **22**, pages 719–722 (2012).
<https://doi.org/10.1017/S0960129511000752>
- [15] Egger, J., R. E. Møgelberg and A. Simpson, *The enriched effect calculus: syntax and semantics*, *Journal of Logic and Computation* **24**, pages 615–654 (2014).
- [16] Ehrhard, T. and G. Guerrieri, *The bang calculus: An untyped lambda-calculus generalizing call-by-name and call-by-value*, in: *Proceedings of the 18th International Symposium on Principles and Practice of Declarative Programming (PPDP'16)*, pages 174–187 (2016).
<https://doi.org/10.1145/2967973.2968608>
- [17] Felleisen, M. and D. Friedman, *Control operators, the SECD-machine, and the λ -calculus*, in: M. Winsing, editor, *Formal Description of Programming Concepts III*, North-Holland (1986).
- [18] Felleisen, M., D. P. Friedman, E. E. Kohlbecker and B. F. Duba, *A syntactic theory of sequential control*, *Theoretical Computer Science* **52**, pages 205–237 (1987).
[https://doi.org/10.1016/0304-3975\(87\)90109-5](https://doi.org/10.1016/0304-3975(87)90109-5)
- [19] Filinski, A., *Representing monads*, in: H. Boehm, B. Lang and D. M. Yellin, editors, *Conference Record of POPL'94: 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 446–457, ACM Press (1994).
<https://doi.org/10.1145/174675.178047>
- [20] Filinski, A., *Controlling Effects*, Ph.D. thesis, School of Computer Science, Carnegie Mellon University (1996).
- [21] Filinski, A., *Towards a comprehensive theory of monadic effects*, in: M. M. T. Chakravarty, Z. Hu and O. Danvy, editors, *Proc. 16th ACM SIGPLAN international conference on Functional Programming, ICFP 2011*, ACM (2011).
<https://doi.org/10.1145/2034773.2034775>
- [22] Gifford, D. K. and J. M. Lucassen, *Integrating functional and imperative programming*, in: W. L. Scherlis, J. H. Williams and R. P. Gabriel, editors, *Proceedings of the 1986 ACM Conference on LISP and Functional Programming, LFP 1986, Cambridge, Massachusetts, USA, August 4-6, 1986*, pages 28–38, ACM (1986).
<https://doi.org/10.1145/319838.319848>
- [23] Griffin, T., *A formulae-as-types notion of control*, in: F. E. Allen, editor, *Conference Record of the Seventeenth Annual ACM Symposium on Principles of Programming Languages, San Francisco (POPL)*, pages 47–58, ACM Press (1990).
<https://doi.org/10.1145/96709.96714>
- [24] Hasegawa, M., *Decomposing typed lambda-calculus into a couple of categorical programming languages*, in: *International Conference on Category Theory and Computer Science* (1995).
- [25] Heijltjes, W., *The Functional Machine Calculus*, in: *Proceedings of the 38th Conference on the Mathematical Foundations of Programming Semantics, MFPS XXXVIII*, volume 1 of *ENTICS* (2022).
<https://doi.org/10.46298/ENTICS.10513>
- [26] Herzberg, D. and T. Reichert, *Concatenative programming: An overlooked paradigm in functional programming*, in: *Proc. 4th International Conference on Software and Data Technologies (ICSOT)*, pages 257–263 (2009).
- [27] Hillerström, D. and S. Lindley, *Shallow effect handlers*, in: S. Ryu, editor, *Proc. Programming Languages and Systems - 16th Asian Symposium (APLAS)*, volume 11275 of *Lecture Notes in Computer Science*, pages 415–435, Springer (2018).
https://doi.org/10.1007/978-3-030-02768-1_22
- [28] Hoare, C. A. R., *Communicating sequential processes*, *Communications of the ACM* **21**, pages 666–677 (1978).
<https://doi.org/10.1145/359576.359585>
- [29] Hughes, J., *Generalising monads to arrows*, *Science of Computer Programming* **37**, pages 67–111 (2000).
- [30] Hyland, M., P. B. Levy, G. Plotkin and J. Power, *Combining algebraic effects with continuations*, *Theoretical Computer Science* **375**, pages 20–40 (2007).
- [31] Hyland, M., G. Plotkin and J. Power, *Combining effects: Sum and tensor*, *Theoretical Computer Science* **357**, pages 70–99 (2006).
- [32] Jones, S. L. P., A. Reid, F. Henderson, C. A. R. Hoare and S. Marlow, *A semantics for imprecise exceptions*, in: B. G. Ryder and B. G. Zorn, editors, *Proc. 1999 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 25–36, ACM (1999).
<https://doi.org/10.1145/301618.301637>

- [33] Jones, S. L. P. and P. Wadler, *Imperative functional programming*, in: M. S. V. Deusen and B. Lang, editors, *Conference Record of the Twentieth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 71–84, ACM Press (1993).
<https://doi.org/10.1145/158511.158524>
- [34] Jouvelot, P. and D. K. Gifford, *Algebraic reconstruction of types and effects*, in: D. S. Wise, editor, *Conference Record of the Eighteenth Annual ACM Symposium on Principles of Programming Languages (POPL)*, pages 303–310, ACM Press (1991).
<https://doi.org/10.1145/99583.99623>
- [35] Kameyama, Y. and M. Sato, *Strong normalizability of the non-deterministic catch/throw calculi*, *Theoretical Computer Science* **272**, pages 223–245 (2002).
[https://doi.org/10.1016/S0304-3975\(00\)00352-2](https://doi.org/10.1016/S0304-3975(00)00352-2)
- [36] Kammar, O., S. Lindley and N. Oury, *Handlers in action*, in: G. Morrisett and T. Uustalu, editors, *ACM SIGPLAN International Conference on Functional Programming (ICFP’13)*, pages 145–158, ACM (2013).
<https://doi.org/10.1145/2500365.2500590>
- [37] Kiselyov, O., A. Sabry and C. Swords, *Extensible effects: an alternative to monad transformers*, in: *Proceedings of the 2013 ACM SIGPLAN symposium on Haskell*, pages 59–70 (2013).
<https://doi.org/10.1145/2503778.2503791>
- [38] Krivine, J.-L., *A call-by-name lambda-calculus machine*, *Higher-Order and Symbolic Computation* **20**, pages 199–207 (2007).
<https://doi.org/10.1007/s10990-007-9018-9>
- [39] Landin, P. J., *The mechanical evaluation of expressions*, *The Computer Journal* **6**, pages 308–320 (1964).
- [40] Landin, P. J., *Correspondence between Algol 60 and Church’s lambda-notation: Part I*, *Communications of the ACM* **8** (1965).
- [41] Landin, P. J., *The next 700 programming languages*, *Communications of the ACM* **9**, pages 157–166 (1966).
<https://doi.org/10.1145/365230.365257>
- [42] Lebresne, S., *A type system for call-by-name exceptions*, *Log. Methods Comput. Sci.* **5** (2009).
[https://doi.org/10.2168/LMCS-5\(4:1\)2009](https://doi.org/10.2168/LMCS-5(4:1)2009)
- [43] Levy, P. B., *Call-by-push-value: A functional/imperative synthesis*, volume 2 of *Semantic Structures in Computation*, Springer Netherlands (2003).
<https://doi.org/10.1007/978-94-007-0954-6>
- [44] Levy, P. B., *Monads and adjunctions for global exceptions*, in: S. D. Brookes and M. W. Mislove, editors, *Proc. 22nd Annual Conference on Mathematical Foundations of Programming Semantics (MFPS)*, volume 158 of *Electronic Notes in Theoretical Computer Science*, pages 261–287, Elsevier (2006).
<https://doi.org/10.1016/J.ENTCS.2006.04.014>
- [45] Levy, P. B., J. Power and H. Thielecke, *Modelling environments in call-by-value programming languages*, *Information and Computation* **185**, pages 182–210 (2003).
- [46] Liang, S., P. Hudak and M. Jones, *Monad transformers and modular interpreters*, in: *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 333–343 (1995).
- [47] Lindley, S., P. Wadler and J. Yallop, *The arrow calculus*, *Journal of Functional Programming* **20**, pages 51–69 (2010).
<https://doi.org/10.1017/S095679680999027X>
- [48] Lüth, C. and N. Ghani, *Composing monads using coproducts*, in: M. Wand and S. L. P. Jones, editors, *Proc. Seventh ACM SIGPLAN International Conference on Functional Programming (ICFP ’02)*, pages 133–144, ACM (2002).
<https://doi.org/10.1145/581478.581492>
- [49] Mac Lane, S., *Categories for the working mathematician*, volume 5 of *Graduate Texts in Mathematics*, Springer New York, New York, second edition (1998).
<https://doi.org/10.1007/978-1-4757-4721-8>
- [50] Maršik, J., M. Amblard and P. de Groote, *Introducing ($|\lambda|$), a λ -calculus for effectful computation*, *Theoretical Computer Science* **869**, pages 108–155 (2021).
- [51] Maurer, L., P. Downen, Z. M. Ariola and S. L. P. Jones, *Compiling without continuations*, in: A. Cohen and M. T. Vechev, editors, *Proc. 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 482–494, ACM (2017).
<https://doi.org/10.1145/3062341.3062380>

- [52] McBride, C. and R. Paterson, *Applicative programming with effects*, Journal of Functional Programming **18**, pages 1–13 (2008).
<https://doi.org/10.1017/S0956796807006326>
- [53] Mifsud, A., R. Milner and A. J. Power, *Control structures*, in: *Proc. 10th Annual IEEE Symposium on Logic in Computer Science (LICS)*, pages 188–198, IEEE Computer Society (1995).
<https://doi.org/10.1109/LICS.1995.523256>
- [54] Mihelič, J., W. Steingartner and V. Novitzká, *A denotational semantics of a concatenative/compositional programming language*, Acta Polytechnica Hungarica **18**, pages 231–250 (2021).
- [55] Milner, R., *A calculus of communicating systems*, volume 92 of *LNCS*, Springer-Verlag, Berlin (1980).
- [56] Milner, R., *Calculi for interaction*, Acta Informatica **33**, pages 707–737 (1996).
- [57] Milner, R., J. Parrow and D. Walker, *A calculus of mobile processes, I*, Information and Computation **100**, pages 1–40 (1992).
[https://doi.org/10.1016/0890-5401\(92\)90008-4](https://doi.org/10.1016/0890-5401(92)90008-4)
- [58] Moggi, E., *Notions of computation and monads*, Information and Computation **93**, pages 55–92 (1991).
- [59] Nielson, F. and H. R. Nielson, *Type and effect systems*, in: E. Olderog and B. Steffen, editors, *Correct System Design, Recent Insight and Advances, (to Hans Langmaack on the occasion of his retirement from his professorship at the University of Kiel)*, volume 1710 of *Lecture Notes in Computer Science*, pages 114–136, Springer (1999).
https://doi.org/10.1007/3-540-48092-7_6
- [60] O’Hearn, P. W., A. J. Power, M. Takeyama and R. D. Tennent, *Syntactic control of interference revisited*, Theoretical Computer Science **228**, pages 211–252 (1999).
[https://doi.org/10.1016/S0304-3975\(98\)00359-4](https://doi.org/10.1016/S0304-3975(98)00359-4)
- [61] Ong, C. L. and C. A. Stewart, *A curry-howard foundation for functional computation with control*, in: P. Lee, F. Henglein and N. D. Jones, editors, *Conference Record of POPL’97: The 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 215–227, ACM Press (1997).
<https://doi.org/10.1145/263699.263722>
- [62] Parigot, M., *$\lambda\mu$ -Calculus: an algorithmic interpretation of classical natural deduction*, in: *International Conference on Logic for Programming Artificial Intelligence and Reasoning (LPAR)*, volume 624 of *Lecture Notes in Computer Science (LNCS)*, pages 190–201 (1992).
- [63] Pestov, S., D. Ehrenberg and J. Groff, *Factor: A dynamic stack-based programming language*, ACM SIGPLAN Notices **45**, pages 43–58 (2010).
<https://doi.org/10.1145/1899661.1869637>
- [64] Peyton-Jones, S. L., *The implementation of functional programming languages*, Prentice Hall (1987).
- [65] Peyton Jones, S. L., A. D. Gordon and S. Finne, *Concurrent haskell*, in: H. Boehm and G. L. S. Jr., editors, *Conference Record of POPL’96: The 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 295–308, ACM Press (1996).
<https://doi.org/10.1145/237721.237794>
- [66] Plotkin, G. and J. Power, *Notions of computation determine monads*, in: *International Conference on Foundations of Software Science and Computation Structures (FoSSaCS)*, pages 342–356, Springer, Berlin, Heidelberg (2002).
- [67] Plotkin, G. and M. Pretnar, *Handling algebraic effects*, Logical Methods in Computer Science (2013).
- [68] Plotkin, G. D., *Call-by-name, call-by-value and the λ -calculus*, Theoretical Computer Science **1**, pages 125–159 (1975).
- [69] Plotkin, G. D. and J. Power, *Adequacy for algebraic effects*, in: F. Honsell and M. Miculan, editors, *Proc. Foundations of Software Science and Computation Structures, 4th International Conference (FOSSACS)*, volume 2030 of *Lecture Notes in Computer Science*, pages 1–24, Springer (2001).
https://doi.org/10.1007/3-540-45315-6_1
- [70] Plotkin, G. D. and J. Power, *Algebraic operations and generic effects*, Applied Categorical Structures **11**, pages 69–94 (2003).
<https://doi.org/10.1023/A:1023064908962>
- [71] Power, A. and H. Thielecke, *Closed Freyd- and κ -categories*, in: *International Colloquium on Automata, Languages, and Programming (ICALP)*, volume 1644 of *LNCS*, pages 625–634, Springer (1999).
- [72] Power, J. and E. Robinson, *Premonoidal categories and notions of computation*, Mathematical Structures in Computer Science **7**, pages 453–468 (1997).
<https://doi.org/10.1017/S0960129597002375>

- [73] Reynolds, J. C., *Syntactic control of interference*, in: A. V. Aho, S. N. Zilles and T. G. Szymanski, editors, *Conference Record of the Fifth Annual ACM Symposium on Principles of Programming Languages (POPL)*, pages 39–46, ACM Press (1978).
<https://doi.org/10.1145/512760.512766>
- [74] Reynolds, J. C., *The discoveries of continuations*, LISP and Symbolic Computation **6**, pages 233–248 (1993).
- [75] Simpson, A. K. and G. D. Plotkin, *Complete axioms for categorical fixed-point operators*, in: *Proc. 15th Annual IEEE Symposium on Logic in Computer Science (LICS)*, pages 30–41, IEEE Computer Society (2000).
<https://doi.org/10.1109/LICS.2000.855753>
- [76] Smetsers, J., E. Barendsen, M. v. Eekelen and M. Plasmeijer, *Guaranteeing safe destructive updates through a type system with uniqueness information for graphs*, in: *Workshop Graph Transformations in Computer Science*, Lecture Notes in Computer Science, Schloss Dagstuhl, Springer-Verlag, Berlin (1993).
- [77] Spivey, J. M., *A functional theory of exceptions*, Science of Computer Programming **14**, pages 25–42 (1990).
[https://doi.org/10.1016/0167-6423\(90\)90056-J](https://doi.org/10.1016/0167-6423(90)90056-J)
- [78] Streicher, T. and B. Reus, *Classical logic, continuation semantics and abstract machines*, Journal of Functional Programming **8**, pages 543–572 (1998).
- [79] Swierstra, W., *Data types à la carte*, Journal of Functional Programming **18**, pages 423–436 (2008).
<https://doi.org/10.1017/S0956796808006758>
- [80] Talpin, J. and P. Jouvelot, *The type and effect discipline*, Information and Computation **111**, pages 245–296 (1994).
<https://doi.org/10.1006/INCO.1994.1046>
- [81] Turner, D. A., *Miranda: A non-strict functional language with polymorphic types*, in: J. Jouannaud, editor, *Proc. Functional Programming Languages and Computer Architecture (FPCA)*, volume 201 of *Lecture Notes in Computer Science*, pages 1–16, Springer (1985).
https://doi.org/10.1007/3-540-15975-4_26
- [82] van Bakel, S., *Exception handling and classical logic*, in: E. Komendantskaya, editor, *Proc. 21st International Symposium on Principles and Practice of Programming Languages (PPDP 2019)*, pages 21:1–21:14, ACM (2019).
<https://doi.org/10.1145/3354166.3354186>
- [83] Wadler, P., *How to replace failure by a list of successes: A method for exception handling, backtracking, and pattern matching in lazy functional languages*, in: J. Jouannaud, editor, *Proc. Functional Programming Languages and Computer Architecture (FPCA 1985)*, volume 201 of *Lecture Notes in Computer Science*, pages 113–128, Springer (1985).
https://doi.org/10.1007/3-540-15975-4_33
- [84] Wadsworth, C. P., *Semantics and Pragmatics of the Lambda-Calculus*, Ph.D. thesis, University of Oxford (1971).
- [85] Wu, N. and T. Schrijvers, *Fusion for free: Efficient algebraic effect handlers*, in: *International Conference on Mathematics of Program Construction* (2015).